# ML24/25-03 Implement anomaly detection sample

Hasan Ahmed
hasan.ahmed@stud.fra-uas.de

Md Abdulla Al Mahamud Rosi
abdulla.rosi@stud.fra-uas.de

***Abstract*— This project introduces a novel anomaly detection system built upon a neuroscience-inspired machine learning approach known as Hierarchical Temporal Memory (HTM). Unlike conventional models, HTM mimics the way the brain processes sequential information, allowing it to identify subtle patterns and irregularities within time-series data. Utilizing the NeoCortex API, the system is designed to handle continuous data streams by encoding temporal and spatial relationships, enabling real-time prediction and anomaly detection. Anomalies are flagged when the difference between expected and observed values exceeds a predefined threshold, signaling unusual behavior. The model is trained using structured numerical sequences and evaluated on real-world datasets to demonstrate its reliability and accuracy. A visual representation of the results is also provided through an interactive graph that highlights detected anomalies. The system operates without the need for labeled data, making it suitable for scenarios where supervised learning is impractical. Its biologically inspired architecture allows for continuous learning, adapting dynamically to changes in data patterns over time. The outcome confirms HTM's capability as an unsupervised, scalable, and domain-independent solution for intelligent anomaly detection.**

***Keywords—Anomaly Detection, HTM, Multisequence Learning, Neocortex API, Machine Learning.***

## I. INTRODUCTION

The concept of Hierarchical Temporal Memory (HTM) emerged from the intersection of neuroscience and artificial intelligence, pioneered by Jeff Hawkins, a visionary in both technology and brain research. Motivated by a deep curiosity about how the human brain—particularly the neocortex—processes information, Hawkins proposed a new approach to machine learning that closely follows biological principles. His foundational ideas were introduced in the 2004 book *"On Intelligence"*, where he explored how intelligent systems could replicate the brain's ability to learn sequences, make predictions, and adapt over time. [1]. It is a Machine Learning approach based on Thousand Brains Theory. This technology operates according to a theory about how the biological neocortex works. Mammals only have one area of the brain called the neocortex, which is responsible for higher order processes like language, conscious movement and thought, and sensory perception [2]. We can think of HTM as a particular type of hierarchical Bayesian model. In order to learn the structure and invariance of the space of problems, it also makes use of spatial-temporal theory [3]. In recent years HTM has also been used for anomaly detection. Something that differs from the expected standard or normal state is called an anomaly. The terms outliers, discordant observations, exceptions, aberrations, surprises, etc. are frequently used to describe the anomalies. Finding anomalous patterns in data is known as anomaly detection. Anomaly detection has been extensively researched and used in many significant domains in recent years, including early warning systems for geological disasters, cyber-intrusion detection, military surveillance, and system fault/damage detection. As related fields develop, more applications in emerging industries like insurance, healthcare, and distributed sensor network surveillance begin to attract attention [4].

By identifying complex temporal patterns and relationships in data, HTM aims to mimic the fundamental workings of the neocortex and predict future events. This methodology excels in simulating sequential learning and related approaches like recurrent neural networks' (RNNs) Gated Recurrent Unit (GRU) and Long Short-Term Memory (LSTM).

A machine learning model called the Hierarchical Temporal Memory Cortical Learning Algorithm (HTM CLA) is intended to replicate the information processing processes of the neocortex in the human brain. Several essential elements are included in this algorithm to handle input data. An encoder first encodes the raw input data, converting it into a sparse distributed representation (SDR). After being run through a spatial pooler, this SDR, which contains information in binary format with few active bits, becomes more resilient to noise. After that, the output is processed by the Temporal Memory component, which oversees identifying and picking up patterns in the data. These acquired patterns are used by the Classifier component to categorize input data and forecast new patterns. Furthermore, the Homeostatic Plasticity Controller guarantees that the system continuously learns new patterns over time [5].

To enable meaningful pattern recognition in HTM systems, raw numeric inputs must first be encoded into a brain-like format that the model can understand. This is where the Scalar Encoder comes in. It converts continuous numerical values into Sparse Distributed Representations (SDRs)—high-dimensional binary vectors with a small number of active bits. The key advantage of this approach is that similar input values generate overlapping SDRs, allowing the HTM system to detect gradual changes, learn temporal patterns, and make informed predictions. The encoder divides the numerical range into overlapping intervals, assigning a unique set of bits for each value to preserve semantic similarity and resist noise. For cyclic inputs, like hours in a day, the Scalar Encoder seamlessly wraps the bit positions to maintain continuity. This encoding process is fundamental to HTM's success in tasks such as anomaly detection and sequence memory, as it prepares data in a way that aligns with the temporal learning mechanisms of the system.

A fundamental element in the In Hierarchical Temporal Memory (HTM), the Spatial Pooler converts raw input into a Sparse Distributed Representation (SDR), ensuring that similar inputs generate overlapping sparse patterns while being resistant to noise. It works through competitive learning, where columns of neurons compete for activation based on input similarity. By activating only a small fraction of columns, the Spatial Pooler maintains efficiency and robustness. Over time, it strengthens connections to frequently occurring patterns, improving the system's ability to generalize. This process is essential for encoding spatial patterns, making it crucial for tasks like sequence learning and anomaly detection in HTM.

## II. METHODOLOGY

In our project, anomaly detection was implemented using the Neocortex API built on the .NET framework, in combination with Hierarchical Temporal Memory (HTM) due to its strong capabilities in learning temporal patterns. We used real numerical datasets for both training and testing, where the test data included intentionally injected anomalies. The HTM model was trained on clean sequences, and then it analyzed modified sequences where the initial part was trimmed to automatically detect anomalies without requiring any manual input.

### *A. Data Set 1:*

For this project, we used two datasets from a weather dataset for Bangladesh. We took temperature of 20 days for different hours.

Below we give our data sequences where the sequences are in JSON files. We keep our dataset in two individual folders which are training_sequence (for training data where 4 files) and predicting_sequence (for predicting data were also 4 files). For example, an hourly sequence of weather temperature of a csv file within training folder.

Below, we provide insight into the structure and content of our data sequences, summarize within these JSON files. This systematic arrangement ensures clarity and accessibility throughout our analysis and model development efforts. Our JSON structure is like the data given below dataset 1:

Listing 1 dataset 1

```
{
38, 40, 46, 29, 33, 27, 42, 47, 30, 31
46, 36, 39, 34, 43, 28, 32, 25, 45, 26
44, 36, 37, 41, 30, 47, 28, 33, 27, 42
37, 46, 26, 47, 44, 36, 29, 42, 38, 27
45, 47, 28, 35, 29, 37, 31, 40, 45, 25
}
```

### *B. Dataset 2:*

We also use another kind of dataset as JSON files (which is fabricated) to predict anomaly detection, where the full process is like the above dataset, we did this to test our HTM process and the accuracy level test for various types of data. The fabricated dataset is based on the temperature of Bangladesh in the year 2025 [7]. Firstly, we keep both training and predicting datasets in output folder in the repository inside this folder, used datasets folders we can use the datasets when we need, we take them from these folders and insert the data into the training_files and predicting_files folders.

Here are also 4 files each file has 5 sequences where each sequence has 10 numerical data (means 10 days temperature value) for training, which is located in the training_sequence folder, and also 4 files each file has 5 sequences where each sequence has 10 numerical data (means 10 days temperature value) for predicting which is located in the predicting_sequence folder. We took a total of 20 days of data. Our Dataset structure is like the data given below dataset 2:

Listing 2 dataset 2 (fabricated) structure

```
{
30, 18, 42, 19, 79, 20, 44, 16, 25, 17
19, 21, 3, 22, 41, 24, 39, 16, 34, 15
43, 32, 27, 35, 45, 30, 41, 23, 18, 31
42, 16, 30, 45, 22, 28, 56, 36, 39, 25
37, 42, 26, 24, 56, 41, 45, 28, 16, 32
}
```

### *C. HTM configuration:*

The process starts with HTM Capturing Configuration and initializing the connection memory then executes the Spatial Pooler and Temporal Memory after that Spatial Pooler Memory is added to the cortex layer and trained for a maximum number of cycles. Since completing the full cycle of numbers, the trained cortex layer and the HTM classifier are returned.

The class requires the configuration settings for both the Encoder and HTM components to be passed to the next level. Then we will utilize the classifier object derived from the trained HTM to make predictions, aiding in anomaly detection.

Currently, we are in the process of preparing training and predicting datasets, which will encompass integer values ranging from 0 to 100. Importantly, these datasets lack any periodic patterns. To facilitate this task effectively, we are adhering to specific settings outlined in a provided listing.

In this configuration, we've chosen to express these integer values using 21 active bits. This decision aims to achieve a balance between precision and efficiency, considering that we're working with 101 distinct values ranging from 0 to 100.

To ensure our encoding scheme effectively covers the entire spectrum of integer values in this dataset, it's essential to compute the total number of input bits needed. This computation, designated as "n," involves adding the count of buckets (101, representing the integer values) to the width of the representation (21 for the active bits), then subtracting one. Thus, the calculation results in $n = \text{buckets} + w - 1 = 101 + 21 - 1 = 121$ [8].

By understanding and implementing these settings, we can effectively encode our data for subsequent training and predicting within our project, ensuring that the representation adequately captures the nuances of the dataset within the specified integer range.

Listing 3 Parameter values set for Encoder

```
int inputBits = 121;
int numColumns = 1210;
Dictionary<string, object> settings = new
Dictionary<string, object>()
        {
           { "W", 21},
           { "N", inputBits}
        };
```

The lower and upper bounds of the data are configured to 0 and 100, respectively, under the assumption that all values will fall within this range. It is important to note that these bounds may need adjustment based on the specific input data being used. We have made some changes to HTM encoder's parameters which is given in above code in Listing 3. We change this because of we faced various issues for this when we run the program it does not works properly and shows some error also then we testing through various values. At last we found these values 121 and 1210 for inputBits and numColumns respectively which works properly for our project [Listing 3].

*D. Code Execution Process:*

In our project, we have two types of datasets so first of all you have to decide which data you want to keep for execute this project. We keep training dataset as primary data inside the folder, "training_sequence", and "predicting_sequence", based on our project directory. To execute our project, clone the project first, then open the terminal write-dent run and press enter then the system asks for a tolerance value from you so give a tolerance value like 0.2 (20 %). Based on this tolerance value our system predicts anomalies for example: if the predicted value is more than 20% then the system declares anomalies.

At the beginning, we use the ExtractSequencesFromFolder method of the CsvSequenceFolder class to read all files inside a folder. These classes maintain a list of numerical sequences from the read data for repeated future use. To handle non-numeric data, exception handling is incorporated within certain classes. Additionally, the TrimSequences technique allows data trimming by removing one to four components (numbers 1 through 4) from the start, returning a numeric sequence [Listing 4].

Listing 4 Sequence Extraction and Trimming Utility for CSV Data

```
public List<List<double>>
ExtractSequencesFromFolder()
      { ....
        return folderSequences;
      }
public static List<List<double>>
TrimSequences(List<List<double>> sequences)
      {....
        return trimmedSequences;
      }
```

Subsequently, the ConvertToHTMInput method of the CSVToHTMInputConverter class is employed to transform all the extracted sequences into a format compatible with Hierarchical Temporal Memory (HTM) training [Listing 5].

Listing 5 Sequence-to-Dictionary Mapping

```
Dictionary<string, List<double>> dictionary = new
Dictionary<string, List<double>>();
for (int i = 0; i < sequences.Count; i++)
   {
   // Unique key created and added to dictionary for
HTM Input
    string key = "S" + (i + 1);
    List<double> value = sequences[i];
    dictionary.Add(key, value);
   }
    return dictionary;
```

After that, the ExecuteHTMModelTraining method of the HTMTrainingManager class is used to train the model using the converted sequences. The numerical data sequences from both the training (for learning) and predicting folders are combined before training the HTM engine. This class then returns the trained model object, which serves as the predictor [Listing 6].

Listing 6 HTM Prediction Initialization

```
.....
MultiSequenceLearning learning = new
MultiSequenceLearning();
predictor = learning.Run(htmInput);
.....
.....
List<List<double>> combinedSequences = new
List<List<double>>(sequences1);
combinedSequences.AddRange(sequences2);
.....
```

In the end, we use the HTMAnomalyExperiment class to detect anomalies in sequences read from files inside the predicting folder. All the previously explained classes - CSV file reading (using CsvSequenceFolder), combining and converting them for HTM training (using CSVToHTMInputConverter), and training the HTM engine (via HTMTrainingManager)—are utilized here. The same CsvSequenceFolder class is used to read files for predicting sequences. The TrimSequences method is then applied to trim sequences for anomaly testing, as explained earlier.fter that we use multisequencelearning classes for training our HTM model which is given below. Where we used the predictor object as a trained HTM model to predict anomalies from our extracted data from predicting_files [Listing 7].

Listing 7 Collecting Testing Data and Anomaly Detection

```
.....
allTestingData.Add(sequence.ToArray());
var (log, anomalies) =
DetectAnomalyOnSequence(predictor,
sequence.ToArray(), _tolerance);
experimentResults.AddRange(log);
allAnomalyIndices.Add(anomalies);
.....
```

The path to the training and predicting folders is set as the default and passed through the constructor, or it can be manually set within the class. [Listing 8].

Listing 8 Configuring CSV Folder Paths

```
.....
_trainingCSVFolderPath =
Path.Combine(projectBaseDirectory, trainingFolderPath);
_predictingCSVFolderPath =
Path.Combine(projectBaseDirectory,
predictingFolderPath);
.....
```

In the end, the DetectAnomaly method is used to detect anomalies in our trimmed sequences one by one, utilizing the trained HTM model predictor [Listing 9].

Listing 9 Applying Anomaly Detection to Test Data

```
foreach (List<double> list in triminputtestseq)
    {
     .....
     double[] lst = list.ToArray();
     DetectAnomaly(myPredictor, lst);
    }
```

Exception handling is implemented to manage errors thrown by the DetectAnomaly method, such as passing non-numeric values or when the number of elements in the list is less than two.

The DetectAnomaly method, which is the main method of the ExtractSequencesFromFolder class, detects anomalies in our data. It traverses each value in the list one by one in a sliding window manner, using the trained model predictor to predict the next element for comparison. An anomaly score is used to quantify the comparison, and if the prediction exceeds a certain tolerance level, it is flagged as an anomaly.

We can obtain our prediction in a list of results in the format of NeoCortexApi.Classifiers.ClassifierResult1[System.String]` from our trained model predictor using the following method [Listing 10].

Listing 10 Predictor to predict Anomalies

```
var res = predictor.Predict(item);
```

In our project, generally we get the output from the HTM is in the following format in our project, when we pass a numerical value 14 for example [Listing 11].

Listing 11 Output looks like given below

```
S3_11-5-12-10-14-13 - 100
S1_5-6-16-10-4-11-7 - 5
.....
```

The first line has the best prediction which the HTM model predicts, with accuracy. We can easily derive the predicted value, which will come after 14 (in this case, it is 13). The string operations are used to get these values. Later we are going to use this to determine anomalies [Listing 12].

Listing 12 Process of anomalies determination

```
if (i < sequence.Length - 1)
   {
       int nextIndex = i + 1;
       double nextItem = sequence[nextIndex];
       double predictedNextItem =
double.Parse(tokens2.Last());

       var anomalyScore =
Math.Abs(predictedNextItem - nextItem);
       var deviation = anomalyScore / nextItem;

       if (deviation <= tolerance)
         {
           resultOutputLines.Add($"No anomaly
detected in the next element. HTM Engine found
similarity: {similarity}%.");
           currentAccuracy += similarity;
         }
       else
         {
           resultOutputLines.Add($"****Anomaly
detected**** in the next element. HTM Engine
predicted: {predictedNextItem} with similarity:
{similarity}%, actual value: {nextItem}.");
           anomalousIndex.Add(i + 1);
           i++; // Skip the anomalous element
           resultOutputLines.Add("Skipping to the next
element in the testing sequence.");
           currentAccuracy += similarity;
         }
   }
```

We are using AnomalyScore, which is nothing, but the absolute value of the ratio of differences between HTM´s predicted number and actual number. If the ratio exceeds tolerance value, we mark it as an anomaly, otherwise, it is not. When an anomaly is detected, we skip that element in the list (we did not pass that value to HTM in the loop).

We use accuracyPerList to record accuracy per numerical sequence tested. record Accuracy is collected from inside each loop run, which indicates HTM model´s accuracy. We also add index positions of anomalies to anomalyIndices, when we encounter indices of anomalies in loop. total Accuracy and list Count is used to calculate average accuracy of the whole experiment.

After that we use static variables to access these from outside, i.e: in Program.cs [Listing 13].

Listing 13 Calculating Total Average Accuracy

```
StoredOutputValues.totalAvgAccuracy =
_totalAccuracy / Math.Max(_iterationCount, 1);
```

The SequenceVisualizer class is used to plot graphs of sequences of data and their anomalies. This class used to show the visual result for detecting anomalies compared to training sequences and the predicting sequence [Listing 14].

Listing 14 Plotting Learned vs. Testing Data

```
public static void
CreateGraphForSequences(List<double[]>
allLearnedData, List<double[]> allTestingData)
.....
allGraphs.Add(actualGraph);
allGraphs.Add(learnedGraph);
.....
var chart = Chart.Plot(allGraphs);
```

Unit Tests: We have developed a unit tests project to test different functionality of this anomaly project. While implementing the unit tests, we ensured the behavior of detection of anomalies in the project. Project files for the units test can be found here.

## III. RESULTS

Once the experiment concludes, the anomaly detection results persisted to the screen, along with HTM accuracy for each individual number sequences and overall HTM accuracy for the whole experiment. We have uploaded the anomaly results of our data in this repository for reference. output result of combined numerical sequence data from training folder (without anomalies) and predicting folder (with anomalies) can be found here.

**Unit test:** We have developed a unit tests project to thoroughly evaluate the functionality of the anomaly detection system. Unit tests are essential for confirming that individual functions and components of the system work as intended in isolation. The primary objective of these tests is to validate that the system behaves as expected across various scenarios, ensuring the accuracy and reliability of anomaly detection. During the implementation of the unit tests, we focused on verifying the system's ability to correctly identify anomalies within given data sequences. Project files for the units test can be found here

**Graphical view:** As part of the anomaly detection project, we incorporated a graphical representation of the training and prediction sequences, along with highlighted anomalies, to facilitate a better understanding of the model's performance. After successfully executing the anomaly detection process, the system generates plots in HTML format, which are displayed directly in the browser for immediate review. Additionally, these plots are saved in the output graph folder for future reference or analysis. We created two types of plots to visualize the results: Training vs. Prediction Plot: This plot compares the actual training data against the predicted values generated by the model. It helps in visually assessing the model's accuracy in predicting normal sequences and detecting patterns (Figure 1). Anomaly Detection Plot: This plot highlights the anomalies detected during the prediction phase. Any data point that deviates significantly from the predicted values is marked, making it easier to spot irregularities in the data (Figure 2). By providing these visualizations, we can gain insights into how well the model performs in real-time anomaly detection and easily identify any areas where improvements may be needed. output graph folder.

1. Training and predicting sequence plot

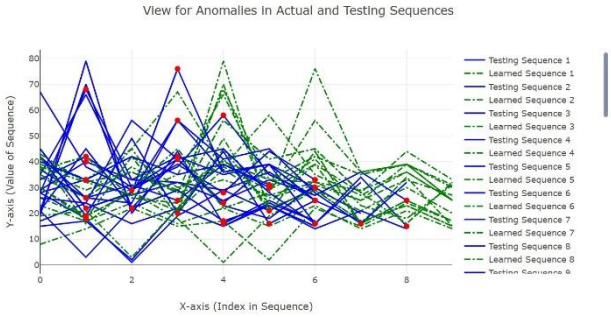


Figure 1: Comparison of Actual vs. Predicted Sequences

2. Training and predicting sequence with anomalies plot

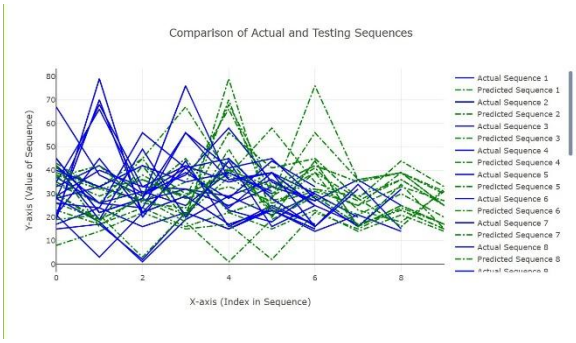


Figure 2: Anomaly Detection in Sequences

We found Anomaly detection results for Testing the sequence: 30, 18, 42, 19, 79, 20, 44, 16, 25, 17

FNR = FN / (FN + TP) = 0/ (0+2) = 0 FPR = FP / (FP + TN) = 2/ (2+5) = 0.29 Where, FN = 0, FP = 2, TN = 5, TP = 2.

After running our sample project, we analysed the output folder of this experiment and got the following average results:

Average FNR of the experiment: 0.22 • Average FPR of the experiment: 0.28

We can observe that the False Negative Rate(FNR) is in our output (0.22). It is desired that the false negative rate should be as lower as possible in an anomaly detection program. Lower false positive rate is also desirable, but not absolutely essential.

Although, it depends on a number of factors, like quantity (the more, the better) and quality of data, and hyperparameters used to tune and train model; more data should be used for training, and hyperparameters should be further tuned to find the most optimal setting for training to get the best results. We

were using less number of numerical sequences as data to demonstrate our sample project due to time and computational constraints, but that can be improved if we use better resources, like cloud.

## IV. Conclusion

Accuracy rate represents the proportion of missed anomalies, where actual anomalies are incorrectly classified as normal. A lower accuracy rate is undesirable, particularly in critical applications like fraud detection. While a higher accuracy rate is generally preferred, it is not always essential, as excessive accuracy may lead to unnecessary investigations and wasted resources. An optimal balance is crucial for an effective anomaly detection model. Hierarchical Temporal Memory (HTM) is well-suited for real-time anomaly detection without requiring prior training and is also resilient to noise in input data.

Our experiment demonstrated varying accuracy levels, influenced by computational limitations. Testing was conducted on a local machine with a smaller dataset due to resource and time constraints. Utilizing more powerful computing resources, such as cloud platforms, can enhance performance. Additionally, increasing the dataset size and fine-tuning hyperparameters can further optimize the model's accuracy.

## References


[1] J. Hawkins and S. Blakeslee, "On Intelligence", Henry Holt, New York, 2004.

[2] J. Struye and S. Latré, "Hierarchical temporal memory and recurrent neural networks for time series prediction: An empirical validation and reduction to multilayer perceptrons," Neurocomputing, vol. 396, pp. 291–301, Jul. 2020, doi: https://doi.org/10.1016/j.neucom.2018.09.098.

[3] J. A. Starzyk, and H. He, "Spatio–Temporal Memories for Machine Learning: A Long-Term Memory Organization", IEEE TRANSACTIONS ON NEURAL NETWORKS, vol. 20, no. 5, pp. 68-780, 2009.

[4] J. Wu, W. Zeng, and F. Yan, "Hierarchical Temporal Memory method for time-series-based anomaly detection," Neurocomputing, vol. 273, pp. 535–546, Jan. 2018, doi: https://doi.org/10.1016/j.neucom.2017.08.026.

[5] "A Machine Learning Guide to HTM (Hierarchical Temporal Memory)," Numenta. https://www.numenta.com/blog/2019/10/24/machine-learning-guide-to-htm/, (Last Accessed: 22.03,2024).